



TRƯỜNG ĐẠI HỌC MỞ TP. HỒ CHÍ MINH
HO CHI MINH CITY OPEN UNIVERSITY

LẬP TRÌNH GIAO DIỆN

Chương 3. Lập trình hướng đối tượng trong C#



1



TRƯỜNG ĐẠI HỌC MỞ TP. HỒ CHÍ MINH
HO CHI MINH CITY OPEN UNIVERSITY

CHUẨN ĐẦU RA

- Phân tích được lợi ích của lập trình hướng đối tượng.
- Mô hình hóa được các đối tượng trong thế giới thực thành các lớp đối tượng trong C#.
- Xây dựng và sử dụng các đối tượng trong lập trình để giải quyết vấn đề.

ThS. Nguyễn Thị Mai Trang

2

2

NỘI DUNG

1. Giới thiệu về Lập trình hướng đối tượng
2. Lớp (class) và các thành phần của lớp
3. Các cách truyền tham số
4. Kế thừa

ThS. Nguyễn Thị Mai Trang

3

3

Chương 3. Lập trình hướng đối tượng trong C#



3.1 Giới thiệu về Lập trình hướng đối tượng

4

3.1 Giới thiệu

- Mục tiêu của việc thiết kế một phần mềm:
 - Tính tái sử dụng (reusability)
 - Tính mở rộng (extensibility)
 - Tính mềm dẻo (flexibility)
- Các cách tiếp cận trong thiết kế
 - Thiết kế theo hàm / thủ tục
 - Thiết kế theo module
 - Thiết kế theo hướng đối tượng

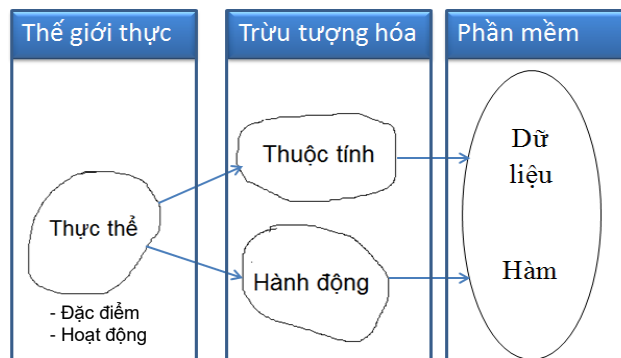
ThS. Nguyễn Thị Mai Trang

5

5

Giới thiệu (tt)

- Thiết kế theo hướng đối tượng



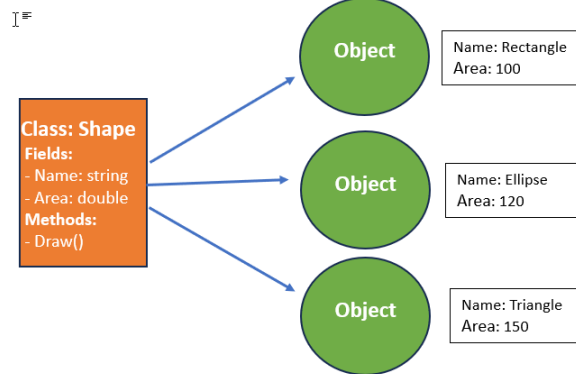
ThS. Nguyễn Thị Mai Trang

6

6

Giới thiệu (tt)

- Lớp (class)
 - Thuộc tính
 - Phương thức
- Đối tượng (object)
 - Định danh
 - Trạng thái
 - Hoạt động



ThS. Nguyễn Thị Mai Trang

7

Giới thiệu (tt)

- Trong ngôn ngữ lập trình, đối tượng là thể hiện của lớp (biến kiểu lớp)
- Giả sử, một lớp có tên là **Hinhtron**, được định nghĩa để lưu trữ kiểu dữ liệu hình tròn
- Trong lớp hình tròn có phương thức **Tinhdientich()** để tính diện tích hình tròn.
- Trong đoạn chương trình bên dưới, biến **ht** chính là đối tượng của lớp **Hinhtron**.

```
Hinhtron ht = new Hinhtron();
double dientich = ht.Tinhdientich();
```

ThS. Nguyễn Thị Mai Trang

8



TRƯỜNG ĐẠI HỌC MỞ TP. HỒ CHÍ MINH
HO CHI MINH CITY OPEN UNIVERSITY

LẬP TRÌNH GIAO DIỆN



Chương 3. Lập trình hướng đối tượng trong C#

3.2 Lớp (class) và các thành phần của lớp

9



TRƯỜNG ĐẠI HỌC MỞ TP. HỒ CHÍ MINH
HO CHI MINH CITY OPEN UNIVERSITY

Lớp (class)

- Lớp (class) là một kiểu cấu trúc mở rộng, được định nghĩa để tạo nên một kiểu dữ liệu mới.
- Một lớp có:
 - dữ liệu
 - thuộc tính
 - phương thức
 - phương thức khởi tạo
 - phương thức tĩnh
 - ...

ThS. Nguyễn Thị Mai Trang

10

10

Lớp (class)

- Khai báo lớp:

```
[MucTruyCap] class TenLop [:LopCoSo]
{
    - Khai báo các trường dữ liệu (các biến)
    - Định nghĩa các phương thức, thuộc tính (properties)
}
```

- Khai báo các thành phần của lớp:

- Biến thuộc lớp: [MucTruyCap] KieuDuLieu TenBien [= GiaTri];

- Phương thức:

```
[MucTruyCap] KieuTraVe TenPhuongThuc ([DanhSachThamSo])
{
    // mã lệnh
}
```

MucTruyCap: public, protected, private (mặc định)

Lớp (class)

- Ví dụ:

```
class Hinhtron
{
    private double bankinh;
    public Hinhtron (double ban_kinh)
    {
        bankinh = ban_kinh;
    }
    public double TinhDientich ()
    {
        return bankinh * bankinh * Math.PI;
    }
}
```

Sử dụng:

```
Hinhtron ht = new Hinhtron(5);
double dientich = ht.TinhDientich();
```

Phương thức (Method)

- Phương thức có trả về giá trị:

```
[MụcTruyCap] kiểu_dulieu TenPhuongthuc ([CacThamSo]) {
    //code
    //return value;
}
```

```
public int Tonghaisho(int a, int b)
{
    return (a + b);
}
```

- Phương thức không có trả về giá trị:

```
[MụcTruyCap] void TenPhuongthuc ([DanhSachThamSo]) {
    //code
}
```

```
public void Tonghaisho(int a, int b)
{
    Console.WriteLine("Tong cua {0} va {1} la {2}", a, b, a + b);
}
```

ThS. Nguyễn Thị Mai Trang

13

13

Các phương thức nạp chồng

- Là các phương thức có cùng tên nhưng khác tham số.
- Được gọi tùy thuộc vào đối số truyền vào.

```
Hinhtron ht1 = new Hinhtron(5);
ht1.TinhDienTich();
```

```
Hinhtron ht1 = new Hinhtron(5);
ht1.TinhDienTich(10);
```

```
class Hinhtron
{
    private double bankinh;
    public Hinhtron (double ban_kinh)
    {
        bankinh = ban_kinh;
    }
    public double TinhDientich ()
    {
        return bankinh * bankinh * Math.PI;
    }
    public double TinhDientich ()
    {
        return bankinh * bankinh * Math.PI;
    }
}
```

ThS. Nguyễn Thị Mai Trang

14

14

Phương thức khởi tạo (Constructor)

- Là phương thức có tên trùng với tên của lớp, được gọi tự động khi một đối tượng của lớp được tạo.
- Thường đặt sau từ khóa public.
- Không có giá trị trả về/void
- Dùng để khởi tạo các giá trị ban đầu cho đối tượng lớp.
- class không có phương thức khởi tạo: trình biên dịch sử dụng phương thức khởi tạo mặc định.
- Các biến không được khởi tạo:
 - Kiểu số: 0
 - Kiểu luận lý: false
 - Kiểu đối tượng: null

ThS. Nguyễn Thị Mai Trang

15

15

Phương thức khởi tạo (Constructor)

```
class Hinhtron
```

```
{
    private double bankinh;
    public Hinhtron()
    {
        bankinh = 1.0;
    }
}
```

Phương thức khởi tạo không tham số

```
class Hinhtron
```

```
{
    private double bankinh;
    public Hinhtron()
    {
        bankinh = 1.0;
    }
    public Hinhtron (double ban_kinh)
    {
        bankinh = ban_kinh;
    }
}
```

Nạp chồng phương thức khởi tạo

ThS. Nguyễn Thị Mai Trang

16

16

Properties

- Các biến dữ liệu của lớp thường được che dấu ở mức độ truy cập private.
- Properties: truy cập, thay đổi giá trị của các trường dữ liệu
- Mỗi property có các mức độ truy cập:
 - get: chỉ cho phép đọc giá trị của trường dữ liệu.
 - set: cho phép gán giá trị cho trường dữ liệu.
 - get và set: cho phép đọc và ghi.

ThS. Nguyễn Thị Mai Trang

17

17

Properties (tt)

- Property chỉ đọc:

```
class Hinhtron
{
    private double bankinh;
    public double Bankinh
    {
        get { return bankinh; }
    }
    public Hinhtron()
    {
        bankinh = 1.0;
    }
}
```

```
Hinhtron ht = new Hinhtron();
double bankinh = ht.Bankinh;
ht.Bankinh = 10; //lỗi cú pháp
```

ThS. Nguyễn Thị Mai Trang

18

18

Properties

•Property chỉ ghi

```
class Hinhtron
{
    private double bankinh;
    public double Bankinh
    {
        set { bankinh = value; }
    }
    public Hinhtron()
    {
        bankinh = 1.0;
    }
}
```

```
Hinhtron ht = new Hinhtron();
ht.Bankinh = 10;
double bankinh = ht.Bankinh; //lỗi cú pháp
```

ThS. Nguyễn Thị Mai Trang

19

19

Properties

•Property đọc và ghi

```
class Hinhtron
{
    private double bankinh;
    public double Bankinh
    {
        get { return bankinh; }
        set { bankinh = value; }
    }
    public Hinhtron()
    {
        bankinh = 1.0;
    }
}
```

```
Hinhtron ht = new Hinhtron();
ht.Bankinh = 10;
double bankinh = ht.Bankinh;
```

ThS. Nguyễn Thị Mai Trang

20

20

Properties

• Lợi ích khi sử dụng Properties:

- Tăng sức mạnh của tính đóng gói
- Bảo vệ dữ liệu, không cho phép gán các giá trị bất hợp lệ.
- Ví dụ:

```
public double Bankinh
{
    get { return bankinh; }
    set
    {
        bankinh = (value < 0 ? 0 : value);
    }
}
```

ThS. Nguyễn Thị Mai Trang

21

21

Dữ liệu và phương thức tĩnh

- Là các biến, phương thức được khai báo sau từ khóa static
- Thường được dùng trong trường hợp cần lưu trữ giá trị các biến hoặc cung cấp các phương thức dùng chung cho các lớp khác nhau trong ứng dụng.
- Truy cập đến dữ liệu và phương thức tĩnh:
 Tenlop.TenBien;
 Tenlop.TenPhuongthuc (...);

```
String.Compare(str1, str2);  
Math.Sqrt(number);
```

ThS. Nguyễn Thị Mai Trang

22

22

TRƯỜNG ĐẠI HỌC MỞ TP. HỒ CHÍ MINH
HO CHI MINH CITY OPEN UNIVERSITY

LẬP TRÌNH GIAO DIỆN

Chương 3. Lập trình hướng đối tượng trong C#



3.3 Các cách truyền tham số

23

TRƯỜNG ĐẠI HỌC MỞ TP. HỒ CHÍ MINH
HO CHI MINH CITY OPEN UNIVERSITY

Các cách truyền tham số

- Truyền bằng tham trị (giá trị):
 - Tương tự C++.
 - Bản sao giá trị của biến đối số được truyền vào phương thức.
- Truyền bằng tham chiếu (reference):
 - Truyền địa chỉ của biến vào làm đối số cho phương thức.
- Có hai cách truyền:
 - truyền tham chiếu với từ khóa ref
 - truyền tham chiếu với từ khóa out.

ThS. Nguyễn Thị Mai Trang

24

24

Các cách truyền tham số (tt)

- **Tham chiếu ref:** sử dụng trong trường hợp biến truyền vào đã được khởi gán giá trị

```
static void Main(string[] args)
{
    int a = 5, b = 10;
    Hoanvi(ref a, ref b);
    //Kết quả: a = 10, b = 5
}
static void Hoanvi (ref int a, ref int b)
{
    int tmp = a;
    a = b;
    b = tmp;
}
```

ThS. Nguyễn Thị Mai Trang

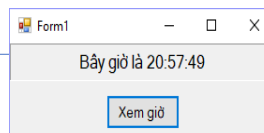
25

25

Các cách truyền tham số (tt)

- **Tham chiếu out:** biến truyền vào không cần gán giá trị trước mà được gán trong phương thức

```
public void LayGioHienTai(out int gio, out int phut, out int giay)
{
    gio = DateTime.Now.Hour;
    phut = DateTime.Now.Minute;
    giay = DateTime.Now.Second;
}
private void btHiengio_Click(object sender, EventArgs e)
{
    int gio, phut, giay;
    LayGioHienTai(out gio, out phut, out giay);
    lbThoigian.Text = String.Format("Bây giờ là {0}:{1}:{2}", gio,
    phut, giay);
}
```



ThS. Nguyễn Thị Mai Trang

26

26



TRƯỜNG ĐẠI HỌC MỞ TP. HỒ CHÍ MINH
HO CHI MINH CITY OPEN UNIVERSITY

LẬP TRÌNH GIAO DIỆN

Chương 3. Lập trình hướng đối tượng với C#



3.4 Kế thừa

27



TRƯỜNG ĐẠI HỌC MỞ TP. HỒ CHÍ MINH
HO CHI MINH CITY OPEN UNIVERSITY

Kế thừa

- Kế thừa phản ánh mối quan hệ giữa các lớp, chia sẻ mã lệnh chương trình giữa các lớp
- Lớp kế thừa (lớp dẫn xuất) là lớp được định nghĩa dựa trên lớp đã có sẵn (lớp cha hay lớp cơ sở).
- Có thể bổ sung thêm các thuộc tính, phương thức.
- Lớp kế thừa có thể sử dụng hầu hết các thành phần giống như lớp cơ sở, ngoại trừ:
 - các thành phần private, static
 - phương thức khởi tạo, phương thức hủy.

ThS. Nguyễn Thị Mai Trang

28

28



TRƯỜNG ĐẠI HỌC MỞ TP. HỒ CHÍ MINH
HO CHI MINH CITY OPEN UNIVERSITY

Kế thừa

• Khai báo:

```
class TenlopKeThua:TenlopCha {
    // khai báo các thành phần
}
```

```
class Diem
{
    protected int x, y;
    public int X {
        get { return x; }
        set { x = value; }
    }
    public int Y {
        get { return y; }
        set { y = value; }
    }
    public string InToado2D()
    {
        return String.Format("C {0} , {1} )", x, y);
    }
}
```

```
class Diem3D : Diem
{
    int z;
    public int Z {
        get { return z; }
        set { z = value; }
    }
    public string InToado3D(){
        return String.Format("C {0} , {1} , {2} )",
            x, y, z);
    }
}
```

```
Diem3D p3 = new Diem3D();
p3.X = 80;
p3.Y = 150;
p3.Z = 50;
string str3D = p3.InToado3D();
```

29

29



TRƯỜNG ĐẠI HỌC MỞ TP. HỒ CHÍ MINH
HO CHI MINH CITY OPEN UNIVERSITY

Kế thừa

• Gọi phương thức khởi tạo của lớp cha

```
TenLopKeThua (ThamSoLopKeThua):base (ThamSoLopCha)
{
    // Khởi tạo cho các thành phần của lớp dẫn xuất
}
```

```
class Diem
{
    protected int x, y;
    ...
    public Diem(int x, int y)
    {
        this.x = x;
        this.y = y;
    }
}
```

```
class Diem3D:Diem
{
    int z;
    ...
    public Diem3D(int x, int y, int z):base(x, y)
    {
        this.z = z;
    }
}
```

30

30

Kế thừa

- Gọi phương thức khởi tạo của lớp cha

```
TenLopKeThua (ThamSoLopKeThua):base (ThamSoLopCha)
{
    // Khởi tạo cho các thành phần của lớp dẫn xuất
}
```

```
class Diem
{
    protected int x, y;
    ...
    public Diem (int x, int y)
    {
        this.x = x;
        this.y = y;
    }
}
```

```
class Diem3D : Diem
{
    int z;
    ...
    public Diem3D(int x, int y, int z)
        : base(x, y)
    {
        this.z = z;
    }
}
```

ThS. Nguyễn Thị Mai Trang

31

31

Kế thừa

- Override phương thức của lớp cha

```
class Polygon
{
    public virtual void Draw()
    {
        Console.WriteLine("Draw polygon");
    }
}
class Triangle:Polygon
{
    public override void Draw()
    {
        Console.WriteLine("Draw Triangle");
    }
}
```

ThS. Nguyễn Thị Mai Trang

32

32



33